КАК ЧИТАТЬ ОТЧЕТ

Подробный разбор HTML/JSON отчета EH/Unwind Analyzer простыми словами

Этот документ объясняет, что означает каждый основной блок отчета: верхние карточки, сведения о PE-файле, структурные проверки, статические признаки, динамические события, план ручной проверки, журнал выполнения и таблица RUNTIME_FUNCTION. Технические термины сохранены там, где их нельзя переводить без потери смысла.

Что дает документ	Как им пользоваться
Понимание отчета сверху вниз	Сначала смотри верхние карточки и итоговую оценку, потом переходи к деталям.
Расшифровку каждого поля	Если непонятна колонка или счетчик, найди ее название в соответствующем разделе.
Подготовку к защите	Можно объяснять преподавателю отчет по этому документу: что найдено, где найдено и что проверять руками.
Границы инструмента	Отчет помогает найти адреса, признаки и события, но не заменяет Ghidra/Debugger для финального смысла кода.

1. Как читать отчет в правильном порядке

Отчет лучше читать не как случайную страницу, а как цепочку доказательств. Верх показывает краткую сводку, середина объясняет структуру PE и найденные признаки, нижние таблицы дают адреса и технические детали.

Шаг	Раздел	Что понять
1	Верхние карточки	Понять общий объем: сколько RUNTIME_FUNCTION, есть ли ошибки разбора, есть ли findings, запускалась ли динамика.
2	Итоговая оценка	Посмотреть verdict, confidence, силу доказательств и есть ли связь между статикой и динамикой.
3	План ручной проверки	Понять, какие действия проект советует выполнить в Ghidra или отладчике.
4	Статические признаки	Посмотреть конкретные finding: код, важность, RVA, секцию и описание.
5	Динамическая проверка	Если запускался --dynamic, найти runtime-события: регистрации handler, вызовы RtlAddFunctionTable, callback и т.д.
6	Таблица RUNTIME_FUNCTION	Перейти к конкретным функциям: Begin/End, Unwind RVA, handler, anomalies/error.
7	Журнал выполнения	Проверить подробный runtime log, если нужно объяснить, откуда взялись динамические счетчики.

Главная идея: отчет не просто ставит оценку. Он показывает маршрут проверки: от общей сводки к конкретным адресам и событиям.

2. Верхний блок отчета

Верхний блок - это панель быстрого обзора. По нему за несколько секунд видно, файл вообще разобрался или нет, есть ли подозрительные признаки, и запускалась ли динамическая проверка.

Поле	Что показывает	Как читать
Путь к файлу	Имя или путь к исследуемому EXE/DLL.	Это объект анализа. Если открыт не тот файл, весь отчет относится не к нужному бинарнику.
Записи RUNTIME_FUNCTION	Сколько записей Windows x64 exception directory найдено.	Много записей - много функций с unwind metadata. Ноль - либо файл простой/leaf-only, либо metadata отсутствует.
Ошибки разбора	Сколько UNWIND_INFO не удалось корректно разобрать.	0 - хорошо. Больше 0 - нужно смотреть таблицу Runtime functions и причину ошибки.
Аномальные записи	Сколько записей имеют аномалии формата UNWIND_INFO.	Аномалии не всегда вредоносны, но требуют проверки.
Эвристические признаки	Сколько finding сформировал движок эвристик.	0 - эвристики не нашли значимых признаков. Больше 0 - смотри раздел Статические признаки.
Оценка подозрительности	Score от 0 до 100.	Это triage-оценка. Чем выше, тем больше вес найденных признаков. Это не вероятность malware.
Статический вывод	Краткий verdict по статике.	Например: сильные статические аномалии не обнаружены, низкая/умеренная/высокая подозрительность.
Уверенность	Confidence в интерпретации статических признаков.	Это надежность вывода по evidence, а не процент вредоносности.
Динамическая проверка	Статус --dynamic.	Не запрашивалась, не запускалась, тайм-аут, ошибка или завершена.

Пример чтения верхних карточек

Если сверху написано: score 0, findings 0, динамическая проверка завершена - значит статически файл выглядит спокойно, но динамический слой все равно мог увидеть runtime-события. Если score высокий и findings много - сначала смотри Статические признаки.

3. Служебные сведения под верхним блоком

Поле	Что означает	Зачем нужно
Сформировано	Дата и время создания отчета.	Помогает понять, свежий ли отчет.
Инструмент / schema	Версия EH/Unwind Analyzer и версия JSON-схемы.	Важно для воспроизводимости и проверки совместимости JSON.
SHA-256	Хэш исследуемого файла.	Доказывает, что отчет относится именно к этому бинарнику, а не к похожей копии.
Сборка	Платформа, компилятор и режим сборки analyzer.	Например windows-x64 / GCC / release. На защите показывает, чем был получен отчет.
Параметры анализа	strict ABI, follow chain, max depth.	Показывает, насколько строгий был decoder и шел ли он по chained unwind records.
Размер файла	Размер исследуемого EXE/DLL в байтах.	Дополнительный контроль объекта анализа.
Команда	Полная команда запуска analyzer.	Главное поле для воспроизводимости: по нему можно повторить отчет.

Параметры strict ABI, follow chain, max depth

strict ABI включает дополнительные проверки соглашения Windows x64 ABI. follow chain разрешает идти по chained UNWIND_INFO. max depth ограничивает глубину переходов, чтобы поврежденный или специально закольцованный файл не загнал анализатор в бесконечный обход.

4. Блок Образ PE

Поле	Простое объяснение	Как использовать
ImageBase	Предпочитаемый базовый адрес PE image.	Для перехода от RVA к VA: VA = ImageBase + RVA, если модуль загружен по базе.
SizeOfImage	Размер образа в памяти.	Все RVA должны попадать внутрь этого диапазона.
SizeOfHeaders	Размер PE-заголовков.	Нужно для проверки, где заканчиваются заголовки и начинаются секции.
Exception directory	RVA и размер таблицы исключений.	На Windows x64 обычно указывает на .pdata с RUNTIME_FUNCTION.
SectionAlignment / FileAlignment	Выравнивание секций в памяти и файле.	Помогает понять различие между RVA и raw offset.

RVA, VA и raw offset

RVA - адрес относительно ImageBase внутри PE image. VA - реальный виртуальный адрес в процессе. Raw offset - смещение байтов в файле на диске. Отчет часто показывает RVA, потому что он стабилен между запусками. В Ghidra обычно надо учитывать ImageBase проекта.

5. Блок Структурная проверка

Поле	Что проверяется	Когда плохо
Raw-диапазоны вне файла	Секция ссылается на байты за пределами файла.	Файл может быть поврежден или специально сделан для поломки парсеров.
Virtual-диапазоны вне image	Секция выходит за SizeOfImage.	RVA такой секции нельзя корректно сопоставить с образом.
Пересечения raw	Raw-области секций пересекаются на диске.	Может быть упаковка, трюк или ошибка структуры.
Пересечения virtual	Секции пересекаются в виртуальном адресном пространстве.	Для нормального PE это подозрительно.
Невыровненные секции	Адреса/размеры не соответствуют alignment.	Может указывать на malformed PE.
Exception directory OOB	Exception directory выходит за допустимые границы.	Таблица RUNTIME_FUNCTION может быть недостоверной.

6. Таблица Секции

Колонка	Что означает	Что смотреть
Имя	Название секции: .text, .pdata, .xdata, .rdata, .data и т.д.	Необычное имя само по себе не приговор, но помогает ориентироваться.
VirtualAddress	RVA начала секции в image.	По нему определяется, в какую секцию попадает RVA.
VirtualSize	Размер секции в памяти.	Показывает, какой диапазон RVA занимает секция.
Raw offset	Смещение секции в файле.	Нужно для hex-viewer и проверки реальных байтов.
Raw size	Размер секции на диске.	Может отличаться от VirtualSize.
Права	R/W/X: read, write, execute.	Для unwind metadata обычно ожидается read-only. Writable или executable metadata требует внимания.

Главное правило: .text обычно executable, .pdata/.xdata обычно read-only. Если UNWIND_INFO лежит в writable-секции, отчет может выдать finding unwind_writable_section.

7. Блок Итоговая оценка

Поле	Что означает	Как не ошибиться
Вывод	Итоговый verdict: например low_suspicion_static или high_suspicion_correlated в русском отображении.	Это краткий вывод, но не заменяет детали ниже.
Сила доказательств	Насколько общий case сильный: нет, слабая, средняя, сильная, с корреляцией или без.	Сильная с корреляцией важнее, чем один изолированный статический признак.
Итоговая уверенность	Обобщенная confidence по статике, динамике и корреляции.	Это не процент вредоносности. Это уверенность модели проекта в своем evidence.
Статика / динамика	Уровни evidence отдельно по статике и runtime.	Например strong / weak означает сильная статика и слабые runtime-сигналы.
Корреляция слоев	Связались ли статические признаки с динамическими событиями.	Да - вывод сильнее. Нет - признаки надо трактовать осторожнее.
Динамические сигналы	Количество категорий runtime-событий.	Это не число строк в логе, а число типов значимых динамических наблюдений.

Что такое correlation

Корреляция означает, что статический признак и динамическое поведение поддерживают друг друга. Например, статически найдены необычные unwind metadata, а динамически программа вызывает RtlAddFunctionTable. Вместе это сильнее, чем каждый факт отдельно.

8. План ручной проверки

План ручной проверки - это не случайный текст. Он формируется автоматически по правилам в report.c. Если analyzer видит конкретный тип finding или dynamic event, он добавляет соответствующую рекомендацию.

Когда появляется рекомендация	Что она простыми словами просит сделать
Проблемы .pdata / диапазонов	Открыть PE viewer, проверить RUNTIME_FUNCTION: begin, end и unwind RVA должны быть внутри SizeOfImage.
Проблемы UNWIND_INFO	Открыть bytes UNWIND_INFO в hex-viewer и сравнить decoded prolog actions с prolog функции в Ghidra.
UNWIND_INFO в необычной секции	Проверить секцию, права доступа и понять: это нормальный compiler layout или ручная подмена/перенаправление.
Handler в non-executable секции	Дизассемблировать handler target и понять, это код, thunk или вообще не-код.
Writable unwind metadata или handler	Понять, оправдано ли это JIT/runtime, или это подозрительное изменение metadata.
Dominant handler target	Открыть handler в Ghidra и посмотреть, почему много функций указывают на одно место.
RtlAddFunctionTable / RtlInstallFunctionTableCallback	Зафиксировать table/count/base/callback и проверить runtime table вместе с module/section mapping.
VEH/VCH/UEF registration	Найти target address зарегистрированного handler и посмотреть его код в Ghidra/отладчике.
Нет динамики, но есть статические findings	Запустить sample с --dynamic на representative exception path, чтобы получить runtime corroboration.

Пример из dynamic-отчета

```
RtlAddFunctionTable table=0x1001ff714 count=1 base=0x7ff6983c0000 RtlInstallFunctionTableCallback
id=0x12345003 base=0x7ff6983c0000 length=4096 callback=0x7ff6983c1460 SetUnhandledExceptionFilter
handler=0x7ff6983c1cd0
```

Простыми словами: программа во время запуска регистрирует runtime function table и обработчик исключений. Отчет показывает адреса. Ручная проверка нужна, чтобы понять, что реально находится по этим адресам.

9. Раздел Статические признаки

Колонка	Что означает	Как читать
Важность	Severity: info, weak, medium, high, critical.	High/critical важнее, но даже они требуют контекста.
Код	Короткий технический идентификатор finding.	Например unwind_writable_section. Его не переводят, чтобы сохранять точность.
Семейство	Группа признака: table_integrity, unwind_integrity, metadata_placement, unwind_reuse, handler_dominance.	Показывает, в какой части формата проблема.
Затронуто	Сколько объектов попало под правило.	Например 46 UNWIND_INFO в writable-секции.
RVA	Представительный адрес, куда можно перейти.	Используется для Ghidra/hex-viewer.
Секция	Секция, в которую попадает representative RVA.	Помогает понять права и размещение.
Описание	Человеческое объяснение finding.	Читай вместе с кодом и RVA.

Семейства признаков

Семейство	Что проверяет
table_integrity	Целостность таблиц PE/.pdata: диапазоны, OOB, overlaps, сортировка, duplicate begin.
unwind_integrity	Корректность UNWIND_INFO: parse failures, bad opcodes, chain problems, ABI warnings.
metadata_placement	Где лежит metadata и handler: необычная, writable, executable или non-executable секция.
unwind_reuse	Повторное использование одного UNWIND_INFO многими функциями.
handler_dominance	Ситуация, когда слишком много функций указывают на один handler target.

Частые finding-коды

Код	Простое объяснение
unwind_writable_section	UNWIND_INFO находится в секции с правом записи. Это необычно для обычного compiler-produced image.
unwind_unusual_section	UNWIND_INFO находится не в ожидаемых .xdata/.rdata/.pdata.
unwind_in_exec_section	Unwind metadata лежит в executable-секции.
handler_nonexec_section	Handler RVA указывает туда, где код не должен исполняться.
handler_writable_section	Handler target находится в writable-секции.
unwind_parse_fail	Decoder не смог разобрать UNWIND_INFO.
unwind_anomalies	UNWIND_INFO разобрался, но внутри есть странности.
invalid_ranges	Begin/End диапазоны RUNTIME_FUNCTION некорректны.
rf_oob / unwind_rva_oob	Function range или UnwindInfoRVA выходит за допустимые границы.
handler_dominance	Один handler обслуживает слишком много функций.

10. Правила снижения уверенности и Детали анализа

Weakening rules нужны, чтобы отчет не звучал слишком уверенно там, где evidence слабое или изолированное. Это важная часть честности проекта: высокий score по одному странному, но объяснимому признаку не должен автоматически превращаться в сильный вывод.

Правило	Что означает
single_signal_only	Вклад дал только один не-сильный признак. Нужна осторожность.
small_sample_size	Мало функций в .pdata, поэтому доли и ratio могут быть завышены.
placement_only_signal	Есть только странное размещение metadata, но нет других подтверждений.
single_family_only	Все признаки из одного семейства. Желательно подтверждение другим слоем.

Детали анализа - это низкоуровневый текстовый журнал статического анализа. Там могут оставаться английские technical details, потому что это близко к внутреннему логированию и помогает сопоставлять отчет с кодом/тестами.

11. Раздел Динамическая проверка

Динамическая проверка появляется, если отчет был создан с флагом --dynamic. Analyzer запускает target под Windows Debug API, ставит breakpoints на важные API/handler и фиксирует runtime evidence.

Карточка	Что показывает	Как читать
Исключения	Сколько exception events увидел debugger.	0 не плохо: sample мог завершиться без исключений.
Попадания в handler	Сколько раз выполнение дошло до handler/VEH/VCH/UEF.	Если больше 0 - есть реальное достижение обработчика.
Вызовы runtime-таблиц	Сумма RtlAddFunctionTable и RtlInstallFunctionTableCallback.	Показывает, что программа регистрировала unwind/function table во время выполнения.
Потерянные события	Сколько событий не удалось сохранить.	0 - хорошо. Больше 0 - runtime evidence неполное.
Инструментация завершена	Удалось ли полностью поставить нужные breakpoints и наблюдение.	Нет не всегда фатально, но снижает полноту динамики.
Handlers найдены / включены	Сколько unwind handlers найдено и сколько удалось поставить под breakpoint.	Если найдено 1 / включено 1 - наблюдение handler успешно.
Ошибки breakpoint	Сколько breakpoints не удалось поставить.	Больше 0 - часть runtime paths могла быть не видна.
Покрытие модулей	Peak modules, failures, truncations.	Показывает, насколько хорошо analyzer видел список модулей процесса.

Таблица динамических событий

Колонка	Что означает
Тип	Тип runtime-события: uef_registration, rtl_add_function_table, rtl_install_function_table_callback, handler hit, process_exit и др.
Модуль	DLL/EXE, где произошло событие: ntdll.dll, kernel32.dll, sample.exe.
Thread	ID потока, в котором событие произошло.
Exception	Код исключения, если событие связано с exception. Для API-регистраций может быть 0.
Chance	first chance или second / n/a. В Windows debugger first chance - первое уведомление об исключении.
Address	Адрес API, handler или точки события.
Arg 0..Arg 3	Аргументы события. Их смысл зависит от Типа.

Как читать Arg 0..Arg 3 для важных событий

Тип события	Arg 0	Arg 1	Arg 2	Arg 3
uef_registration	handler address	-	-	-
veh_registration	handler address	first/flags	-	-
vch_registration	handler address	first/flags	-	-
rtl_add_function_table	table address	count	base address	-
rtl_install_function_table_callback	table identifier	base address	length	callback address
handler hit	context/address details	зависит от события	зависит от события	зависит от события
process_exit	exit code	-	-	-

12. Журнал выполнения

Журнал выполнения - это подробный текстовый след динамической проверки. Если таблица событий дает компактные строки, то журнал показывает, как analyzer к ним пришел.

Строка в журнале	Что значит простыми словами
CREATE_PROCESS	Target был запущен под debugger, указан PID и base address.
API ... resolved	Analyzer нашел адрес интересующей Windows API в модуле.
BP set ... OK	Breakpoint поставлен успешно.
SetUnhandledExceptionFilter handler=...	Программа зарегистрировала UEF handler по указанному адресу.
RtlAddFunctionTable table=... count=... base=...	Программа добавила runtime function table.
RtlInstallFunctionTableCallback id=... callback=...	Программа зарегистрировала callback, который может выдавать function table по запросу runtime.
handler hit	Выполнение реально дошло до handler.
EXIT_PROCESS	Процесс завершился, указан exit code.

Если на защите спрашивают, откуда взялись динамические выводы, показывай сначала карточки, затем таблицу событий, затем соответствующие строки в журнале.

13. Таблица RUNTIME_FUNCTION

Это самый большой и технический раздел отчета. Он показывает каждую запись runtime function table и связанный с ней UNWIND_INFO.

Колонка	Что означает	Как использовать
#	Номер строки в отчете.	Удобно ссылаться на конкретную запись.
Begin	RVA начала диапазона функции.	В Ghidra можно перейти к ImageBase + Begin.
End	RVA конца диапазона функции.	Диапазон функции по metadata: [Begin, End).
Size	End - Begin.	Нулевой или странный размер может быть признаком ошибки.
Секция кода	Секция, куда попадает Begin.	Обычно .text. Если не .text - надо смотреть контекст.
Unwind RVA	RVA структуры UNWIND_INFO.	Адрес metadata, которую Windows использует для unwind.
Unwind section	Секция, где лежит UNWIND_INFO.	Обычно .xdata/.rdata. Writable/executable/unusual - повод проверить.

Колонка	Что означает	Как использовать
Статус	OK или ошибка разбора.	Ошибка означает, что UNWIND_INFO не удалось корректно декодировать.
Version / flags	Версия UNWIND_INFO и flags: EHANDLER, UHANDLER, CHAININFO.	Флаги говорят, есть ли handler или chain.
Stack	Сколько stack bytes описано unwind actions.	Помогает сопоставлять с prolog функции.
Handler	RVA обработчика, если он есть.	Переходи туда в Ghidra, если finding связан с handler.
Аномалии / ошибка	Краткий список parse/anomaly markers.	Если не '-' - надо смотреть подробнее.

Поля UNWIND_INFO внутри JSON

Поле JSON	Смысл
version	Версия структуры UNWIND_INFO.
flags	Числовые flags UNWIND_INFO.
prolog_size	Размер prolog функции, который описывает unwind metadata.
count_codes	Количество unwind code записей.
fixed_stack_bytes	Оценка фиксированного изменения stack.
frame_reg / frame_off	Frame register и offset, если используется frame pointer.
has_handler / handler_rva / handler_section	Есть ли exception handler и куда он указывает.
has_chain / chain	Есть ли chained RUNTIME_FUNCTION и куда он ведет.
anomalies / anomalies_text	Битовая маска и текстовое описание аномалий.

14. Что находится в JSON-отчете

HTML нужен человеку. JSON нужен для автоматической проверки, тестов, скриптов и воспроизводимости. Ключи JSON специально оставлены на английском, чтобы не ломать schema и интеграции.

Верхний ключ	Что хранит
input	Путь к исследуемому файлу.
report_metadata	Версия схемы, версия инструмента, hash, команда запуска и параметры анализа.
image_base, size_of_image, size_of_headers	Основные поля PE image.
section_alignment, file_alignment	Выравнивание секций.
exception_directory	RVA и size exception directory.
pe_validation	Счетчики структурных проблем PE.
sections	Массив секций PE.
heuristic_analysis	Статический score, verdict, counters, findings, weakening rules.
dynamic_validation	Динамические события и счетчики, если запускался --dynamic.
assessment	Итоговая оценка и план ручной проверки.
functions	Массив записей RUNTIME_FUNCTION и decoded UNWIND_INFO.

Heuristic analysis: важные поля

Поле	Что означает
score / suspicion	Score 0..100 и его нормализованный вариант 0..1.
confidence	Уверенность в статической интерпретации.
verdict	Статический verdict.
requires_manual_review	Нужно ли ручное подтверждение.
total	Всего RUNTIME_FUNCTION записей.
rf_invalid_ranges, rf_oob, rf_unwind_oob, rf_unsorted, rf_overlaps, rf_duplicate_begins	Счетчики проблем таблицы RUNTIME_FUNCTION.
unwind_parse_fail, unwind_anomalies, unwind_abi_warnings	Счетчики проблем UNWIND_INFO и ABI.
unwind_unusual_section, unwind_exec_section, unwind_writable_section	Где необычно размещены unwind metadata.
handler_nonexec_section, handler_writable_section	Проблемы размещения handler target.
unwind_reuse_ratio, unwind_unique_ratio, handler_reuse_ratio	Доли повторного использования metadata/handler.
evidence_families, evidence_level	Какие семейства evidence сработали и насколько они сильны.
findings	Список конкретных признаков.
weakening_rules_applied	Список правил, которые снизили уверенность.

Dynamic validation: важные поля

Поле	Что означает
enabled, ran, ok, timed_out	Была ли динамика включена, запускалась ли, завершилась ли успешно, был ли timeout.
target_exited, target_exit_code	Завершился ли target и с каким кодом.
exceptions_seen, first_chance_seen, second_chance_seen	Счетчики exception events.
veh_registrations, vch_registrations, uef_set_calls	Регистрации VEH/VCH/UEF.
veh_hits, vch_hits, uef_hits, unwind_handler_hits	Попадания в обработчики.
rtl_add_function_table_calls, rtl_install_function_table_cb_calls	Вызовы runtime function table API.
dynamic_signal_count	Сколько типов runtime-сигналов найдено.
dropped_events	Сколько событий потеряно.
events	Табличные события с адресами и аргументами.
details	Текстовый журнал выполнения.
trace	Дополнительная трасса адресов, если есть.

15. Как использовать отчет вместе с Ghidra

Отчет дает адреса и секции, но смысл кода часто надо смотреть в Ghidra. Главный мост между отчетом и Ghidra - RVA.

Что есть в отчете	Что делать в Ghidra
Begin RVA функции	Перейти к адресу ImageBase + Begin RVA и сравнить границы функции.
Unwind RVA	Найти область UNWIND_INFO, если надо проверять metadata bytes.
Handler RVA	Перейти к handler и посмотреть disassembly/decompiler.
Representative RVA finding	Перейти к конкретному месту, на которое ссылается finding.
Section permissions	Сравнить Memory Map Ghidra с таблицей секций отчета.
Ghidra CSV	Импортировать CSV, если нужно проставить имена/комментарии по runtime functions.

16. Типовые сценарии чтения

Ситуация	Что делать
Все чисто: score 0, findings 0	Показать верхние карточки, Статические признаки и таблицу Runtime functions с OK-строками.
Есть static finding	Открыть Статические признаки, выписать code, severity, affected, RVA, section, затем перейти в Runtime functions/Ghidra.
Есть dynamic events	Открыть Динамическую проверку, найти Type и Arg 0..3, затем показать Журнал выполнения.
UNWIND_INFO в writable-секции	Открыть Секции, проверить права, потом Runtime functions с этим Unwind RVA.
Handler target подозрительный	Перейти по Handler RVA в Ghidra и объяснить, код это или нет.
Timeout или instrumentation incomplete	Сказать, что динамика неполная; статические findings остаются, но runtime evidence ограничено.

17. Мини-глоссарий

Термин	Простое объяснение
PE	Формат EXE/DLL в Windows.
PE image	То, как PE-файл отображается в память процесса.
RVA	Адрес относительно ImageBase.
VA	Реальный виртуальный адрес в процессе.
raw offset	Смещение байтов в файле на диске.
.pdata	Секция, где обычно лежит таблица RUNTIME_FUNCTION.
.xdata	Секция, где обычно лежит UNWIND_INFO.
RUNTIME_FUNCTION	Запись диапазона функции: BeginAddress, EndAddress, UnwindInfoAddress.
UNWIND_INFO	Metadata, которая описывает, как восстановить stack/registers при exception/unwind.
handler	Код-обработчик исключения или unwind-события.
VEH	Vectored Exception Handler.
VCH	Vectored Continue Handler.
UEF	Unhandled Exception Filter.
RtlAddFunctionTable	Windows API для регистрации runtime function table.
RtlInstallFunctionTableCallback	Windows API для регистрации callback, который предоставляет function table.
finding	Структурированный признак, найденный эвристикой.
score	Числовая оценка подозрительности 0..100.
confidence	Уверенность в интерпретации evidence.
correlation	Ситуация, когда статические и динамические признаки поддерживают друг друга.

18. Короткая шпаргалка для защиты

Если нужно быстро объяснить отчет, говори так:

• Верхние карточки дают общий обзор: сколько функций, есть ли ошибки, findings, score и запускалась ли динамика.

• Образ PE и Секции показывают, где лежат данные и какие у секций права доступа.

• Итоговая оценка сжимает статические и динамические наблюдения в verdict, confidence и case strength.

• Статические признаки показывают конкретные правила, которые сработали, с адресом RVA и секцией.

• Динамическая проверка показывает runtime-события: регистрации handler, вызовы RtlAddFunctionTable, callback, попадания в обработчики.

• План ручной проверки объясняет, что именно нужно подтвердить в Ghidra или отладчике.

• Таблица RUNTIME_FUNCTION дает техническую основу: диапазон функции, Unwind RVA, handler, flags, stack и ошибки.

• JSON хранит те же данные для автоматической проверки и тестов, поэтому ключи JSON остаются английскими.

Главная фраза: отчет показывает не только итоговый verdict, но и проверяемую цепочку evidence: адреса, секции, findings, runtime-события и ручной план проверки.

ДОПОЛНЕНИЕ: СТАТИЧЕСКИЙ ОТЧЕТ

Разбор каждого блока и поля статического HTML/JSON отчета

Статический отчет - это результат анализа файла без запуска target под Windows Debug API. Он отвечает на вопрос: что записано в PE-файле, как устроены секции, корректна ли таблица RUNTIME_FUNCTION, как декодируется UNWIND_INFO и какие эвристические признаки сработали.

Что важно	Простыми словами
Статика не исполняет программу	Она читает байты файла, PE-заголовки, секции, exception directory, RUNTIME_FUNCTION и UNWIND_INFO.
Динамический блок не раскрывается	Вверху остается карточка статуса динамики, но отдельного раздела событий/журнала нет, если не был указан --dynamic.
Вывод строится по evidence	Score, verdict и confidence берутся из эвристик в heuristics.c и итоговой сборки assessment в report.c.
Ручной план не случайный	Он формируется функцией collect_manual_validation_steps по конкретным findings, семействам признаков и weakening rules.

Кодовые точки проекта: HTML делает report_write_html, JSON делает report_write_json_ex, признаки считает heuristics_analyze, план ручной проверки собирает collect_manual_validation_steps.

19. Как читать именно статический отчет

Статический отчет лучше читать как цепочку: сначала общий статус, потом структура PE, затем признаки, после этого адресная таблица RUNTIME_FUNCTION. Не начинай с большой таблицы функций: без верхних карточек и PE-блоков трудно понять, что в ней искать.

Шаг	Куда смотреть	Что понять
1	Верхние карточки	Сколько найдено функций, есть ли ошибки parse, сколько findings, какой score/verdict/confidence.
2	Служебные сведения	Какой файл анализировали, какой SHA-256, какой командой и с какими флагами.
3	Образ PE и структурная проверка	Не сломан ли PE-контейнер: секции, границы, выравнивание, exception directory.
4	Секции	Где лежат .text/.pdata/.xdata/.rdata и какие у них права R/W/X.
5	Итоговая оценка	Как движок сжал static evidence в final verdict и почему просит/не просит ручную проверку.
6	Статические признаки	Какие конкретные правила сработали: code, family, severity, affected, RVA, section.
7	Детали анализа	Низкоуровневый текстовый след: счетчики, notes, mitigation, диагностические строки.
8	Таблица RUNTIME_FUNCTION	Проверить конкретные диапазоны, Unwind RVA, flags, handler, stack и parse/anomaly error.

20. Чем статический HTML отличается от dynamic/combo отчета

Элемент	В статическом отчете	В dynamic/combo отчете
Карточка Динамическая проверка	Обычно показывает: не запрашивалась. Это нормально.	Показывает завершена/ошибка/тайм-аут и влияет на runtime evidence.
Раздел Динамическая проверка	Не выводится, потому что dr отсутствует.	Выводятся карточки runtime counters, таблица событий и журнал выполнения.
runtime_evidence_level	none, потому что нет запуска.	weak/medium/strong, если были runtime-сигналы и сессия достаточно полная.
static_dynamic_correlation	Нет, потому что сравнивать статические признаки с runtime events нечем.	Может стать да, если слои подтверждают друг друга.
manual_validation_steps	Часто советует открыть Ghidra/PE viewer или запустить --dynamic для подтверждения.	Может советовать проверить конкретные registrations, handlers и runtime table calls.

Главное: статический отчет не доказывает, что конкретная ветка кода реально выполнится. Он показывает структуру файла и признаки, которые надо проверить адресно.

21. Верхний блок статического HTML

Верхний блок создается всегда. Если анализ запускался с --heuristics, в нем появляется полный набор карточек по эвристикам.

Элемент	Что выводится	Как читать
EH/Unwind Analyzer	Заголовок отчета.	Показывает, что это человекочитаемый HTML dashboard, а не JSON или CSV.
Путь под заголовком	input_path исследуемого EXE/DLL.	Проверь, что открыт отчет именно по нужному бинарнику.
Записи RUNTIME_FUNCTION	Количество записей из exception directory.	0 может быть нормально для leaf-only/resource-only, но для сложного x64 кода это повод проверить .pdata.
Ошибки разбора	Сколько UNWIND_INFO не удалось корректно декодировать.	0 хорошо. Больше 0 - смотри таблицу RUNTIME_FUNCTION и finding unwind_parse_fail.
Аномальные записи	Сколько UNWIND_INFO имеют anomalies: версия, opcode, chain, ABI и т.п.	Не равно malware, но это место для ручной проверки.
Эвристические признаки	Количество findings из heuristic_analysis.findings.	Если больше 0, обязательно читай раздел Статические признаки.
Оценка подозрительности	score 0..100.	Это triage score по правилам проекта, а не процент вредоносности.
Статический вывод	verdict по статике: нет сильных аномалий, низкая, умеренная или высокая подозрительность.	Это краткий вывод по static evidence до ручной проверки.
Уверенность	confidence 0..100%.	Надежность интерпретации evidence. Может снижаться weakening rules.
Динамическая проверка	Статус dynamic слоя.	В чистом static-отчете обычно: не запрашивалась.

Служебные поля под карточками

Поле	Что это	Зачем нужно
Сформировано	Время генерации отчета.	Отличает свежий отчет от старого.
Инструмент / schema	Версия analyzer и версия JSON schema.	Важно для воспроизводимости и проверки совместимости.
SHA-256	Хэш входного файла.	Доказывает, что отчет относится именно к этому бинарнику.
Сборка	Платформа, компилятор, режим debug/release.	Показывает, каким analyzer был собран.
Параметры анализа	strict ABI, follow chain, max depth.	Объясняет строгость декодера и обработку chained UNWIND_INFO.
Размер файла	input_size в байтах.	Быстрая sanity-check проверка объекта анализа.
Команда	Команда запуска analyzer.	По ней можно повторить отчет. Аргументы после -- редактируются/маскируются по правилам проекта.

22. PE-блоки статического отчета

Образ PE

Поле	Простое объяснение	Как использовать
ImageBase	Предпочитаемый базовый адрес PE image.	Для перехода из RVA к VA: VA = ImageBase + RVA, если модуль загружен по базе.
SizeOfImage	Размер PE-образа в памяти.	Begin/End/Unwind RVA должны попадать внутрь этого диапазона.
SizeOfHeaders	Размер заголовков PE.	Помогает понять, где заканчиваются headers и начинаются section data.
Exception directory	RVA + размер директории исключений.	Для x64 обычно указывает на .pdata с таблицей RUNTIME_FUNCTION.
SectionAlignment / FileAlignment	Выравнивание секций в памяти и на диске.	Если нарушено, RVA/raw mapping может быть ненадежным.

Структурная проверка

Поле	Что проверяется	Когда плохо
Raw-диапазоны вне файла	Секция ссылается на байты за пределами файла.	Файл может быть поврежден, упакован или намеренно malformed.
Virtual-диапазоны вне image	Секция выходит за SizeOfImage.	RVA такой секции нельзя надежно сопоставить с PE image.
Пересечения raw	Raw-области секций пересекаются на диске.	Может быть packing trick или ошибка заголовков.
Пересечения virtual	Секции пересекаются в виртуальном адресном пространстве.	Для нормального PE это подозрительно.
Невыровненные секции	Адреса/размеры нарушают alignment.	Может ломать корректный разбор и перевод RVA -> raw.
Exception directory OOB	Exception directory неполная или выходит за допустимые границы.	Таблица RUNTIME_FUNCTION может быть недостоверной.

Таблица Секции

Колонка	Что означает	Что смотреть
Имя	Имя секции из PE header: .text, .pdata, .xdata, .rdata и т.п.	Необычное имя само по себе не приговор, но помогает искать metadata placement.
VirtualAddress	RVA начала секции в image.	По нему отчет определяет, в какую секцию попадает Begin/Unwind/Handler RVA.
VirtualSize	Размер секции в памяти.	Показывает, какой RVA-диапазон занимает секция.
Raw offset	Смещение секции в файле.	Нужно для hex viewer и проверки реальных байтов.
Raw size	Размер секции на диске.	Может отличаться от VirtualSize.
Права	R/W/X: read, write, execute.	Для UNWIND_INFO обычно ожидается read-only; writable/executable metadata требует проверки.

23. Итоговая оценка в статическом отчете

Раздел Итоговая оценка появляется, когда есть heuristics или dynamic data. В чистом static-отчете он опирается на heuristic_analysis и показывает, что динамический слой не дал evidence.

Карточка	Что показывает	Как читать в static-режиме
Вывод	final_verdict из assessment.	Обычно *_static или no_strong_correlated_evidence. Это финальный текстовый вывод.
Сила доказательств	case_strength.	Без динамики сильная статика обычно strong_unconfirmed_case, а не correlated case.

Карточка	Что показывает	Как читать в static-режиме
Итоговая уверенность	final_confidence.	Берется из confidence с учетом итоговой логики assessment.
Статика / динамика	static_evidence_level / runtime_evidence_level.	В static отчете runtime обычно none.
Корреляция слоев	static_dynamic_correlation.	В static отчете обычно нет, потому что нет runtime слоя.
Динамические сигналы	dynamic_signal_count.	В static отчете 0.

Значения verdict и case strength

Значение	Простыми словами
no_strong_correlated_evidence	Нет сильной связанной цепочки evidence.
low_suspicion_static	Есть слабые/низкие статические признаки без runtime-подтверждения.
moderate_suspicion_static	Есть умеренная статическая подозрительность, нужна ручная проверка.
high_suspicion_static	Статические признаки сильные, но без динамики это все еще unconfirmed.
*_correlated	Статика и динамика поддержали друг друга. В чистом static-отчете такого обычно нет.
runtime_signals_only	Есть runtime-сигналы без значимой статики. Для чистого static-отчета не характерно.
no_case	Нет meaningful evidence.
weak_case / medium_case	Evidence есть, но не сильное или не подтвержденное другим слоем.
strong_unconfirmed_case	Сильная статика есть, но dynamic corroboration отсутствует.
medium_correlated_case / strong_correlated_case	Есть связь static и dynamic evidence.

План ручной проверки в static-отчете

План формируется автоматически в report.c не по текстовой фантазии, а по конкретным счетчикам, findings и семействам evidence.

Когда появляется	Что рекомендует сделать
Проблемы .pdata / диапазонов	Открыть PE viewer, отсортировать RUNTIME_FUNCTION и проверить Begin/End/Unwind RVA внутри SizeOfImage.
unwind_parse_fail или unwind_anomalies	Открыть bytes UNWIND_INFO в hex-viewer и сравнить decoded prolog actions с прологом функции в Ghidra.
metadata_placement	Проверить секцию с unwind metadata: RVA, raw offset, права доступа и нормальность layout.
handler_nonexec_section	Дизассемблировать handler target и понять, это код, thunk или не-code bytes.
writable metadata/handler	Проверить, оправдано ли это JIT/runtime сценарием; иначе считать размещение подозрительным.
handler_dominance	Открыть dominant handler target в Ghidra и сравнить несколько функций, которые на него ссылаются.
weakening rules	Сначала исключить доброкачественные объяснения, потому что уверенность специально снижена.
Есть static findings, но нет динамики	Запустить sample повторно с --dynamic на representative exception path для runtime corroboration.

24. Раздел Статические признаки

Это центральный раздел static-отчета. Он показывает не просто общий score, а конкретные правила, которые сработали.

Колонка	Что означает	Как читать
Важность	severity: info, weak, medium, high, critical.	High/critical важнее, но даже они требуют контекста и ручной проверки.
Код	Технический идентификатор finding.	По нему можно связать HTML, JSON, тест и правило в heuristics.c.
Семейство	Группа evidence.	Показывает, какой слой проблемы: таблица, unwind, размещение, reuse, handler dominance.

Колонка	Что означает	Как читать
Затронуто	Сколько объектов попало под правило.	1 - точечный случай; большое число - системный паттерн.
RVA	Representative RVA.	Адрес, с которого удобно начать ручную проверку в Ghidra.
Секция	Секция, куда попадает representative RVA.	Помогает понять права и назначение области.
Описание	Русское пояснение finding.	Кратко говорит, что именно показалось необычным.

Семейства evidence

Family	Что означает
table_integrity	Целостность PE/.pdata/RUNTIME_FUNCTION: границы, сортировка, пересечения, дубликаты, exception directory.
unwind_integrity	Корректность UNWIND_INFO: parse errors, anomalies, chain/opcode/version/ABI проблемы.
metadata_placement	Где лежит unwind metadata или handler: необычная, writable, executable, non-executable/unknown секция.
unwind_reuse	Слишком много функций ссылаются на один и тот же UnwindInfoRVA, особенно вместе с другими сигналами.
handler_dominance	Много обработчиков указывает на один handler target, что может быть runtime helper или dispatcher.

Каталог finding code

Code	Family	Что значит простыми словами
pe_section_raw_oob	table_integrity	Raw-диапазон секции выходит за размер файла.
pe_section_virtual_oob	table_integrity	Virtual-диапазон секции выходит за SizeOfImage.
pe_section_raw_overlap	table_integrity	Raw-области секций пересекаются на диске.
pe_section_virtual_overlap	table_integrity	Virtual-области секций пересекаются в памяти.
pe_section_misaligned	table_integrity	Нарушены PE alignment/header invariants или правила выравнивания секций.
pe_exception_directory_oob	table_integrity	Exception directory повреждена, неполная, нечитаемая или вне границ.
no_runtime_functions	table_integrity	Не найдено RUNTIME_FUNCTION. Может быть benign leaf-only, но надо подтвердить сложность кода.
invalid_ranges	table_integrity	Есть записи, где End <= Begin.
rf_oob	table_integrity	Begin/End выходят за SizeOfImage.
unwind_rva_oob	table_integrity	UnwindInfoRVA отсутствует или выходит за допустимые границы.
rf_unsorted	table_integrity	Таблица не отсортирована по BeginAddress.
rf_overlaps	table_integrity	Диапазоны функций пересекаются после сортировки.
duplicate_begin	table_integrity	Несколько записей имеют одинаковый BeginAddress.
unwind_parse_fail	unwind_integrity	UNWIND_INFO не удалось декодировать.
unwind_anomalies	unwind_integrity	UNWIND_INFO декодируется, но имеет аномалии версии/opcode/chain/ABI.
placement_integrity_combo	unwind_integrity	Подозрительное размещение metadata подтверждается проблемами UNWIND_INFO или UnwindInfoRVA.
unwind_in_exec_section	metadata_placement	UNWIND_INFO лежит в executable-секции.
unwind_unusual_section	metadata_placement	UNWIND_INFO лежит не в привычных .xdata/.rdata/.pdata.
unwind_writable_section	metadata_placement	UNWIND_INFO лежит в writable-секции.
handler_nonexec_section	metadata_placement	Handler RVA указывает в non-executable или unknown секцию.
handler_writable_section	metadata_placement	Handler RVA указывает в writable-секцию.
placement_exec_nonexec_handler_combo	metadata_placement	Unwind metadata в executable-секции сочетается с handler в non-executable/unknown секции.
writable_metadata_handler_combo	metadata_placement	Writable unwind metadata сочетается с writable handler target.
unwind_reuse_strong	unwind_reuse	Большая доля функций ссылается на один UnwindInfoRVA.
unwind_reuse_moderate	unwind_reuse	Умеренный reuse подтверждается другим статическим сигналом.
reuse_dominance_combo	unwind_reuse	Heavy UnwindInfo reuse совпал с dominant handler target.
handler_dominance	handler_dominance	Один handler target доминирует среди многих функций.

25. Правила снижения уверенности и Детали анализа

Weakening rules нужны, чтобы отчет не звучал слишком уверенно, когда evidence узкое, одиночное или имеет доброкачественные объяснения. Они уменьшают confidence, но не стирают finding.

Code	Когда применяется	Что это значит
single_signal_only	Только один не-сильный признак.	Как standalone evidence это слабое основание для вывода.
small_sample_size	В .pdata меньше 32 записей и нет strong findings.	Малая выборка может завысить доли и ratios.
placement_only_signal	Есть только необычное размещение metadata без других integrity/handler проблем.	Нужно проверить, не является ли это редким, но нормальным layout.
single_family_only	Evidence только из одного семейства и нет strong findings.	Перед сильным выводом желательно подтверждение другим слоем.

Детали анализа

Элемент	Что там находится	Как использовать
pre-блок Детали анализа	Текстовый журнал heuristics: notes, counters, mitigation lines, диагностические пояснения.	Если преподаватель спрашивает, откуда взялся score, этот блок показывает внутренний след решения.
notes	Нейтральные наблюдения, например отсутствие RUNTIME_FUNCTION в resource-only образе.	Не каждое сообщение является подозрением.
mitigation/...	Строки о снижении уверенности.	Показывают честность модели: проект сам говорит, где evidence слабее.
raw technical details	Часть текста может оставаться с английскими терминами.	Это нормально: строки близки к внутренним тестам и JSON-контракту.

26. Таблица RUNTIME_FUNCTION в статическом отчете

Это самая адресная часть static-отчета. Она показывает каждую запись exception directory и связанный с ней UNWIND_INFO.

Элемент	Что означает	Как читать
Фильтр по RVA, секции, флагу или ошибке	Поле поиска над таблицей.	Введи RVA, .xdata, EHANDLER, Ошибка и таблица покажет только совпадения.
Счетчик N из M	Сколько строк видно после фильтра.	Помогает понимать, сколько записей осталось на экране.
#	Индекс записи.	Удобно ссылаться на строку отчета.
Begin	RVA начала диапазона функции.	В Ghidra переходи к ImageBase + Begin.
End	RVA конца диапазона функции.	Диапазон функции по metadata: [Begin, End).
Size	End - Begin.	Нулевой/странный размер часто связан с invalid_ranges.
Секция кода	Секция, куда попал Begin.	Обычно .text. Если не .text, смотри контекст.
Unwind RVA	RVA структуры UNWIND_INFO.	Ключевой адрес metadata для проверки в .xdata/.rdata/.pdata.
Unwind section	Секция, где лежит UNWIND_INFO.	Writable/executable/unusual секция - повод проверить.
Статус	OK или Ошибка.	Ошибка значит, что decoder не смог корректно разобрать UNWIND_INFO.
Version / flags	Версия и флаги UNWIND_INFO.	EHANDLER/UHANDLER означает наличие handler; CHAININFO - chained metadata.
Stack	fixed_stack_bytes.	Сколько байт stack описали unwind actions.
Handler	RVA handler, если он есть.	Если указан, переходи в Ghidra и смотри реальный код target.
Аномалии / ошибка	anomalies_text и parse_error.	Главное поле для объяснения, почему строка подозрительна или не разобралась.

Поля UNWIND_INFO внутри JSON

Поле	Смысл
version	Версия структуры UNWIND_INFO.
flags	Числовые flags: handler, chained info и т.п.
prolog_size	Размер пролога функции, который описан unwind metadata.
count_codes	Количество unwind code entries.
fixed_stack_bytes	Оценка фиксированного stack allocation по decoded actions.
frame_reg / frame_off	Frame register и offset, если функция использует frame pointer схему.
has_handler / handler_rva / handler_section	Есть ли exception handler, его RVA и секция.
has_chain / chain	Есть ли chained RUNTIME_FUNCTION и какие Begin/End/Unwind RVA у цепочки.
anomalies / anomalies_text	Количество и текст anomalies.

27. JSON static report: какие ключи соответствуют HTML

HTML нужен для человека, JSON - для тестов, схемы и автоматической проверки. В static JSON нет dynamic_validation, если запуск был без --dynamic.

JSON-ключ	Где это видно в HTML	Что означает
input	Путь под заголовком	Файл, который анализировали.
report_metadata	Служебные сведения	Версия, SHA-256, команда, build, флаги анализа.
image_base	Образ PE	ImageBase.
size_of_image	Образ PE	SizeOfImage.
size_of_headers	Образ PE	SizeOfHeaders.
section_alignment / file_alignment	Образ PE	Выравнивание секций в памяти и файле.
exception_directory	Образ PE	rva и size директории исключений.
pe_validation	Структурная проверка	Счетчики structural PE problems.
sections	Секции	Массив секций с адресами, raw offsets, sizes, flags R/W/X.
heuristic_analysis	Верхние карточки, Статические признаки, Детали анализа	Все static counters, findings, score, confidence, weakening rules.
assessment	Итоговая оценка и План ручной проверки	Сжатый вывод и manual_validation_steps.
functions	Таблица RUNTIME_FUNCTION	Каждая запись Begin/End/Unwind + декодированный UNWIND_INFO.

Основные поля heuristic_analysis

Группа полей	Поля	Смысл
Итог	score, suspicion, confidence, requires_manual_review, verdict, evidence_level	Общая оценка статического слоя.
Сила findings	weak_finding_count, medium_finding_count, strong_finding_count, total	Сколько признаков и сколько записей всего анализировалось.
PE counters	pe_section_raw_oob, pe_section_virtual_oob, pe_section_raw_overlaps, pe_section_virtual_overlaps, pe_section_misaligned, pe_exception_directory_oob	Повторяют структурные проблемы PE.
RUNTIME_FUNCTION counters	rf_invalid_ranges, rf_oob, rf_unwind_oob, rf_unsorted, rf_overlaps, rf_duplicate_begins	Ошибки диапазонов, сортировки, границ и дубликатов.
UNWIND counters	unwind_parse_fail, unwind_anomalies, unwind_abi_warnings	Проблемы декодирования и ABI.
Placement counters	unwind_unusual_section, unwind_exec_section, unwind_writable_section, handler_nonexec_section, handler_writable_section	Где лежит metadata/handler и насколько это нормально.
Reuse counters	unwind_reuse_ratio, unwind_unique_ratio, handler_count, handler_reuse_ratio	Повторное использование UNWIND_INFO и handler targets.
Evidence	evidence_family_count, evidence_families, findings	Какие семьи evidence представлены и какие findings сработали.
Ослабление	weakening_rules_applied, weakening_rule_count	Почему confidence снижен.
details	details	Текстовый диагностический журнал.

Поля report_metadata

Поле	Что хранит
schema_version	Версия JSON schema, сейчас 1.1.0.
tool_version	Версия EH/Unwind Analyzer.
generated_at_utc	Время создания отчета в UTC.
input_sha256	SHA-256 анализируемого файла.
input_size	Размер файла в байтах.
command_line	Команда запуска analyzer.
build_compiler / build_platform / build_mode	Чем и как собран analyzer.
strict_abi / follow_chain / max_chain_depth	Параметры декодирования и анализа chained metadata.

28. Как объяснять статический отчет на защите

Ситуация	Что сказать простыми словами
score 0, findings 0	Статика не нашла значимых нарушений PE/unwind metadata; это не доказывает отсутствие вредоносности вообще, но по этой области все спокойно.
Есть unwind_parse_fail	Часть UNWIND_INFO не декодируется. Надо открыть конкретный Unwind RVA и проверить bytes/структуру.
Есть handler_nonexec_section	Handler RVA указывает туда, где код выполняться не должен. Надо перейти по адресу и понять, это ошибка, thunk, packing или искусственный переход.
Есть writable metadata	UNWIND_INFO или handler находятся в writable-секции. Это может быть JIT/runtime, но для обычного PE требует объяснения.
Есть handler_dominance	Много функций указывают на один handler. Это может быть общий runtime helper или централизованный dispatcher.
Есть weakening rules	Проект сам снижает уверенность, потому что evidence одиночное, маленькое или из одного семейства.
Нет dynamic_validation	Отчет статический. Для поведения во время запуска нужен отдельный запуск с --dynamic.

Короткая формула: статический отчет показывает проверяемые адреса и structural evidence; окончательный смысл подозрительных строк подтверждается в Ghidra, PE viewer и при необходимости dynamic запуском.

ПОЧЕМУ АНОМАЛЬНЫЕ ЗАПИСИ НЕ РАВНЫ ВСЕМ ПОДОЗРЕНИЯМ

Этот раздел нужен, чтобы правильно объяснять верхнюю карточку `Аномальные записи`. На первый взгляд может показаться, что она должна считать все подозрения в отчете. На самом деле это был бы методически неправильный счетчик: разные подозрения имеют разные единицы измерения. Поэтому карточка считает только локальные аномалии конкретных записей `RUNTIME_FUNCTION/UNWIND_INFO`, а общий уровень подозрительности выводится отдельными блоками.

29. Что именно считает карточка `Аномальные записи`

Карточка находится в самом верхнем блоке HTML-отчета рядом с `Записи RUNTIME_FUNCTION` и `Ошибки разбора`. Ее смысл привязан к таблице `RUNTIME_FUNCTION`. Для каждой строки таблицы analyzer берет `Unwind RVA`, читает соответствующую структуру `UNWIND_INFO` и пытается ее декодировать. Если при декодировании у этой строки появился флаг `anomalies`, строка попадает в счетчик `Аномальные записи`.

```
Упрощенная логика верхней карточки:
parse_failures = количество строк, где UNWIND_INFO не разобрался
anomaly_count = количество строк, где us[i].anomalies != 0
карточка `Аномальные записи` показывает anomaly_count
```

Что считается	Простыми словами	Пример
Строка RUNTIME_FUNCTION	Одна запись из exception directory: Begin, End, Unwind RVA.	Например строка #7 в нижней таблице.
UNWIND_INFO этой строки	Metadata, которая описывает, как восстанавливать stack/registers при exception/unwind.	version, flags, prolog_size, unwind codes, handler.
anomalies	Признаки странности внутри декодированного UNWIND_INFO.	Плохой opcode, странные operands, chain/version/ABI anomaly.
Аномальные записи	Сколько строк таблицы имеют такие anomalies.	Если 1 строка имеет bad opcode, карточка покажет 1.

30. Почему это не счетчик всех подозрений

В отчете есть несколько разных классов evidence. Они не равны друг другу и не должны складываться в одну простую цифру. Одни признаки считаются по строкам таблицы, другие по глобальному паттерну, третьи по runtime-событиям. Если сложить их в одну карточку, число станет внешне удобным, но потеряет смысл.

Тип evidence	Единица измерения	Почему не кладется в `Аномальные записи`
UNWIND_INFO anomalies	Строка `RUNTIME_FUNCTION`.	Это как раз локальная аномалия строки, поэтому она входит в карточку.
unwind_parse_fail	Строка, где UNWIND_INFO не разобрался.	Это отдельная карточка `Ошибки разбора`, потому что decode failed - не то же самое, что decoded but anomalous.
unwind_reuse	Ratio: сколько функций используют один UnwindInfoRVA.	Это глобальный паттерн reuse, а не отдельная ошибка одной строки.
handler_dominance	Паттерн: много функций указывают на один handler.	Это распределение ссылок, а не anomaly bits у конкретного UNWIND_INFO.
writable/unusual/executable section	Размещение metadata или handler в секции.	Это проблема placement, она отображается в `Статические признаки`.
combo findings	Комбинация нескольких признаков.	Если добавить их в тот же счетчик, получится двойной счет.
dynamic events	События во время запуска процесса.	Это runtime evidence, а не строка статической таблицы файла.

31. Почему такой подход правильный

Правильный отчет не должен превращать разные типы evidence в одну неясную цифру. В инженерной проверке важно сохранять смысл каждого счетчика: что именно считали, по какой единице, где это можно проверить руками и какой вывод из этого допустим.

Причина	Подробное объяснение
Сохраняется единица измерения	`Аномальные записи` - это count строк. `handler_dominance` - это pattern/ratio. `dynamic events` - это события исполнения. Они не одного типа.
Избегается ложный масштаб	Один finding `handler_dominance` может затрагивать 261 функцию. Если назвать это `261 аномальная запись`, будет неясно, это 261 независимая ошибка или один общий паттерн.
Избегается двойной счет	Комбинированный finding вроде `reuse_dominance_combo` уже строится поверх reuse и dominance. Если складывать все вместе, один и тот же факт усилит счет несколько раз.
Сохраняется проверяемость	Для `Аномальных записей` можно сразу открыть таблицу RUNTIME_FUNCTION и перейти к конкретным строкам. Для dominance/reuse проверка другая.
Отчет остается explainable	Комиссии можно показать: верхние карточки дают быстрые разные срезы, а не одну мутную сумму всего подряд.

32. Где тогда смотреть все подозрения

Все подозрения не теряются. Они просто разложены по правильным блокам. Если нужно понять общую картину, нельзя смотреть только на `Аномальные записи`. Нужно читать связку верхних карточек и подробных разделов.

Блок отчета	Что показывает	Когда смотреть
Аномальные записи	Локальные anomalies внутри UNWIND_INFO у строк RUNTIME_FUNCTION.	Когда нужно понять, есть ли битые/странные unwind metadata entries.
Ошибки разбора	Сколько UNWIND_INFO вообще не удалось корректно разобрать.	Когда decoder не смог прочитать metadata.
Эвристические признаки	Сколько сработало правил heuristics.	Это главный быстрый счетчик всех static findings.
Оценка подозрительности	Score 0..100 по правилам проекта.	Быстрая оценка силы static evidence.
Статический вывод	Краткий verdict по статике.	Когда нужен простой итог по static analysis.
Статические признаки	Таблица всех findings: code, family, severity, affected, RVA, section, описание.	Главное место, где перечислены все подозрения по статике.
Динамическая проверка	Runtime-события: registrations, RtlAddFunctionTable, callbacks, handler hits и т.д.	Когда отчет создан с `--dynamic`.
Итоговая оценка	Final verdict, confidence, static/runtime evidence level, correlation.	Когда нужно объяснить общий вывод отчета.

33. Примеры, почему ноль в `Аномальные записи` может быть нормальным

Ноль в этой карточке означает только одно: analyzer не нашел строк `RUNTIME_FUNCTION`, у которых выставлены `UNWIND_INFO anomalies`. Это не означает, что весь отчет чистый. Другие признаки могут быть ненулевыми.

Sample	Аномальные записи	Другие признаки	Как объяснять
07_static_unwind_bad_opcode.exe	Не 0	unwind_parse_fail, unwind_anomalies	Есть реальные anomalies в UNWIND_INFO, карточка срабатывает.
08_static_unwind_bad_operands.exe	Не 0	unwind_anomalies	Проблема именно в операндах/кодах UNWIND_INFO.
14_static_reuse_dominance_combo.exe	0	unwind_reuse_strong, handler_dominance, reuse_dominance_combo	UNWIND_INFO формально декодируется, но глобальный паттерн reuse/dominance подозрителен.
18_dynamic_rtl_function_table.exe	0	dynamic runtime table calls	Статика спокойная, зато dynamic layer фиксирует RtlAddFunctionTable/Callback.

Именно поэтому нельзя говорить: `Аномальные записи = 0, значит подозрений нет`. Правильно говорить: `локальных UNWIND_INFO anomalies нет, но общий вывод надо смотреть по Эвристическим признакам,

Статическим признакам и динамике`.

34. Как объяснять комиссии

Готовая формулировка для защиты:

Карточка `Аномальные записи` не является общим счетчиком подозрительности. Она показывает только количество строк `RUNTIME_FUNCTION`, у которых при декодировании связанного `UNWIND_INFO` обнаружены anomalies. Остальные подозрения имеют другую природу: placement, reuse, handler dominance, combo findings или runtime events. Поэтому они отображаются отдельно в `Эвристических признаках`, `Статических признаках`, `Оценке подозрительности` и `Динамической проверке`. Такой подход нужен, чтобы не смешивать разные единицы измерения и не получать двойной счет.

Если комиссия спросит: `Почему не считать все подозрения одной цифрой?`

Ответ: потому что это были бы разные типы фактов. Строка с bad opcode - это локальная структурная аномалия. Dominant handler - это глобальный паттерн распределения ссылок. Dynamic RtlAddFunctionTable - это событие runtime. Их нельзя честно складывать как одинаковые предметы. Для общей оценки есть score, verdict и таблица findings.

Если комиссия спросит: `Значит ли 0, что файл чистый?`

Ответ: нет. Ноль означает только отсутствие локальных `UNWIND_INFO anomalies` в строках `RUNTIME_FUNCTION`. Файл все равно может иметь findings другого типа: unusual section, writable metadata, reuse, dominance, dynamic registrations. Поэтому отчет читается не по одной карточке, а по цепочке evidence.

35. Нужно ли менять логику карточки

Логику карточки менять на `считать все подозрения` не стоит. Это ухудшит объяснимость отчета. Правильнее считать общие подозрения через уже существующие поля: `Эвристические признаки`, `Оценка подозрительности`, `Статические признаки`, `Итоговая оценка` и `Динамическая проверка`.

Вариант	Плюс	Минус	Вывод
Оставить текущую логику	Счетчик остается точным и проверяемым по строкам.	Название может требовать пояснения.	Методически правильно.
Складывать все подозрения в карточку	На первый взгляд проще.	Смешивает строки, ratios, patterns, combo findings и runtime events.	Методически хуже.
Пояснять или переименовать	Снимает двусмысленность.	Нужно привыкнуть к точному названию.	Лучший путь для защиты.

Самое точное название по смыслу: `Аномалии UNWIND_INFO` или `Аномальные UNWIND-записи`. Даже если в HTML остается короткая подпись `Аномальные записи`, в защите ее надо читать именно так: `аномальные записи unwind metadata`, а не `все подозрения проекта`.

36. Подробная расшифровка каждой верхней карточки

Верхние карточки - это не замена всему отчету, а быстрый triage. Они отвечают на вопрос: `куда смотреть сначала`. Каждая карточка имеет свой смысл и свою единицу измерения. Ошибка новичка - читать одну карточку как окончательный вывод. Правильно читать их вместе.

Карточка	Что считает	Что означает 0	Что означает значение больше 0
Записи RUNTIME_FUNCTION	Количество записей exception directory / .pdata.	Таблица функций не найдена. Для leaf-only или resource-only это может быть нормально.	Есть функции с unwind metadata. Чем больше число, тем богаче материал для анализа.
Ошибки разбора	Сколько UNWIND_INFO не удалось декодировать.	Decoder смог прочитать все найденные UNWIND_INFO.	Часть metadata повреждена, нестандартна или указывает не туда. Смотреть `Статические признаки` и строки со статусом `Ошибка`.
Аномальные записи	Сколько строк имеют `us[i].anomalies`.	Нет локальных anomalies у декодированных unwind entries.	Есть строки с подозрительными opcode/operands/version/chain/ABI markers. Это не все подозрения, а только этот слой.

Карточка	Что считает	Что означает 0	Что означает значение больше 0
Эвристические признаки	Сколько findings сформировал heuristics engine.	Эвристики не нашли значимых static findings.	Есть хотя бы одно правило, которое считает ситуацию требующей внимания.
Оценка подозрительности	Score 0..100 по статическим правилам.	Статика спокойная по текущей модели.	Чем выше score, тем сильнее static evidence. Это не процент malware.
Статический вывод	Краткий verdict по static evidence.	Может быть `нет сильных аномалий`.	Показывает уровень подозрительности: low/moderate/high.
Уверенность	Confidence в интерпретации evidence.	Низкая уверенность не значит чисто, а значит evidence слабое или ограниченное.	Высокая уверенность значит, что evidence шире, сильнее или лучше подтверждено.
Динамическая проверка	Статус dynamic layer.	Если не запрашивалась, это нормально для static-отчета.	Если dynamic включена, смотреть runtime counters, events и журнал.

37. Как связаны карточки между собой

Карточки надо читать как цепочку. Например, `Аномальные записи = 0`, но `Эвристические признаки = 3` - это не противоречие. Это означает: локальных anomalies UNWIND_INFO нет, но есть другие static findings, например reuse/dominance/placement. Другой пример: `Ошибки разбора = 1`, `Аномальные записи = 0` - значит decoder вообще не смог разобрать часть metadata; это вынесено в отдельный счетчик parse failures.

Комбинация карточек	Как читать
Записи RUNTIME_FUNCTION > 0, Ошибки = 0, Аномальные = 0, Findings = 0	По EH/unwind metadata все выглядит спокойно. Это не доказывает, что файл безопасен вообще, но по этой области сильных сигналов нет.
Аномальные = 0, Findings > 0	Подозрения есть, но они не являются локальными anomalies UNWIND_INFO. Ищи их в `Статические признаки`.
Ошибки разбора > 0	Часть UNWIND_INFO не прочиталась. Это отдельный серьезный повод для ручной проверки.
Score высокий, а Аномальные = 0	Score набрали другие families: table_integrity, metadata_placement, unwind_reuse, handler_dominance или combo findings.
Динамическая проверка завершена, static score = 0	Статика спокойная, но runtime layer мог зафиксировать поведение: registrations, runtime table calls, handler hits.
Корреляция слоев = да	Статический и динамический слои поддерживают друг друга. Это сильнее, чем один слой отдельно.

38. Служебные элементы под верхним блоком

Служебные поля нужны для воспроизводимости. На защите они показывают, что отчет не просто картинка, а документ с проверяемым источником: какой файл, какой hash, какая команда, какая версия analyzer.

Элемент	Что это	Как объяснить комиссии
Путь к файлу	Input binary, который анализировали.	Отчет относится именно к этому EXE/DLL, а не к другому sample.
Сформировано	Дата и время генерации отчета.	Можно отличить свежий отчет от старого.
Инструмент / schema	Версия analyzer и версия JSON schema.	Показывает контракт отчета и совместимость полей.
SHA-256	Криптографический hash input file.	Если файл изменился хотя бы на байт, hash будет другим.
Сборка	Платформа, компилятор, debug/release.	Показывает, каким analyzer был собран.
Параметры анализа	strict ABI, follow chain, max depth.	Объясняет строгость decoding и обработку chained UNWIND_INFO.
Размер файла	Размер input в байтах.	Простая sanity-check проверка объекта.
Команда	Полная команда запуска analyzer.	По ней можно повторить тот же отчет.

39. Образ PE и структурная проверка

Эти блоки отвечают не за malware behavior, а за качество контейнера PE и адресную модель. Если здесь проблемы, все последующие адреса надо читать осторожнее, потому что section mapping может быть нарушен.

Элемент	Смысл	Что проверять
ImageBase	Предпочтительная база загрузки PE image.	Для перехода от RVA к VA: VA = ImageBase + RVA, если модуль загружен по базе.
SizeOfImage	Размер PE image в памяти.	Begin/End/Unwind RVA должны попадать внутрь этого диапазона.
SizeOfHeaders	Размер PE headers.	Помогает понять, где заканчиваются headers и начинаются sections.
Exception directory	RVA и size таблицы exception/unwind metadata.	Для Windows x64 обычно ведет к .pdata с RUNTIME_FUNCTION.
SectionAlignment / FileAlignment	Выравнивание в памяти и на диске.	Нарушения могут ломать перевод RVA <-> raw offset.
Raw-диапазоны вне файла	Секция ссылается на байты за пределами файла.	Проверять PE headers и raw offsets.
Virtual-диапазоны вне image	Секция выходит за SizeOfImage.	RVA mapping может быть недостоверен.
Пересечения raw/virtual	Секции перекрываются.	Может быть packing trick, ошибка или malformed PE.
Exception directory OOB	Директория исключений выходит за допустимые границы.	Таблица RUNTIME_FUNCTION может быть неполной или ложной.

40. Таблица `Секции`: почему она важна

Секции связывают адреса отчета с реальными областями PE. Без понимания секций невозможно правильно объяснить, почему `unwind_writable_section` или `handler_nonexec_section` важны.

Колонка	Что означает	Как читать
Имя	Имя секции: .text, .pdata, .xdata, .rdata и т.п.	Имя помогает понять назначение области, но не является абсолютным доказательством.
VirtualAddress	RVA начала секции в памяти.	По нему определяется, куда попал Begin/Unwind/Handler RVA.
VirtualSize	Размер секции в памяти.	Показывает виртуальный диапазон секции.
Raw offset	Смещение секции в файле.	Нужно для проверки байтов в hex viewer.
Raw size	Размер секции на диске.	Может отличаться от VirtualSize.
Права R/W/X	Read, write, execute.	UNWIND_INFO обычно ожидается read-only. Writable/executable metadata требует объяснения.

41. Итоговая оценка: что означает каждый элемент

Итоговая оценка собирает static evidence, dynamic evidence и weakening rules в один общий вывод. Это не магический classifier, а объяснимая сводка того, что уже найдено в деталях.

Элемент	Что показывает	Как говорить на защите
Вывод / final_verdict	Краткий итог: no strong evidence, low/moderate/high static, correlated, runtime only.	Это итоговая формулировка, но ее надо подтверждать findings и events.
Сила доказательств / case_strength	Насколько сильна цепочка evidence.	Strong unconfirmed - сильная статика без динамики. Correlated - статика и динамика поддержали друг друга.
Итоговая уверенность	Надежность интерпретации evidence.	Это не вероятность malware, а confidence в выводе analyzer.
Статика / динамика	Уровень evidence по каждому слою: none/weak/medium/strong.	Показывает, какой слой внес основной вклад.
Корреляция слоев	Поддерживают ли static и runtime друг друга.	Если да, вывод сильнее, потому что есть cross-layer corroboration.
Динамические сигналы	Сколько runtime signal units найдено.	Например registrations, runtime table calls, handler hits.

Элемент	Что показывает	Как говорить на защите
План ручной проверки	Список действий для Ghidra/debugger.	Это не случайный текст: он формируется по сработавшим findings/events.

42. Статические признаки: как читать каждую колонку

Если верхние карточки говорят, что есть findings, главный подробный блок - `Статические признаки`. Именно здесь перечислены конкретные правила, а не только общий score.

Колонка	Что означает	Как использовать
Важность	Severity finding: info, weak, medium, high, critical.	High важнее, но любой finding надо проверять контекстом.
Код	Технический идентификатор правила.	По нему можно связать HTML, JSON, tests и код heuristics.c.
Семейство	Группа evidence: table_integrity, unwind_integrity, metadata_placement, unwind_reuse, handler_dominance.	Показывает, в каком слое проблема.
Затронуто	Сколько объектов попало под правило.	1 - точечный случай. Большое число - широкий паттерн.
RVA	Representative RVA для начала проверки.	Переход в Ghidra: ImageBase + RVA.
Секция	Секция representative RVA.	Помогает понять права и назначение области.
Описание	Человеческое пояснение finding.	Кратко объясняет, почему правило сработало.

43. Динамическая проверка: что значит каждый элемент

Динамический раздел появляется только при запуске с `--dynamic`. Он не заменяет статику, а показывает, что sample реально сделал во время запуска под Windows Debug API.

Элемент	Что означает	Как читать
Исключения	Количество exception events, увиденных debugger.	0 не всегда плохо: sample мог завершиться без exception path.
Попадания в handler	Сколько раз выполнение дошло до unwind/VEH/VCH/UEF handler.	Если больше 0, есть runtime подтверждение достижения handler.
Вызовы runtime-таблиц	RtlAddFunctionTable + RtlInstallFunctionTableCallback.	Показывает runtime регистрацию function table.
Потерянные события	События, которые не удалось сохранить.	Больше 0 снижает полноту динамического evidence.
Инструментация завершена	Удалось ли поставить нужные breakpoints и провести наблюдение.	Нет не всегда фатально, но evidence ограничено.
Handlers найдены / включены	Сколько handler targets найдено статикой и сколько поставлено на breakpoint.	Показывает покрытие handler observation.
Ошибки breakpoint	Неудачные постановки breakpoint.	Если больше 0, часть runtime paths могла быть не видна.
Покрытие модулей	Peak modules, refresh failures, truncations.	Показывает, насколько analyzer видел module mapping процесса.
Таблица событий	Type, module, thread, exception, chance, address, arg0..arg3.	Главное место для доказательства конкретных runtime calls/registrations.
Журнал выполнения	Подробный trace/debug log.	Используется, если надо объяснить, откуда взялись counters.

44. Таблица `RUNTIME_FUNCTION`: подробная карта строк

Нижняя таблица - самый адресный технический блок. Она нужна для перехода от общей оценки к проверяемым строкам. Если комиссия просит показать `где именно`, ответ обычно начинается здесь.

Колонка	Что означает	Что проверять руками
#	Номер строки в отчете.	Удобно ссылаться на конкретную запись.
Begin	RVA начала функции.	Перейти в Ghidra по ImageBase + Begin.
End	RVA конца функции.	Проверить, что диапазон разумный и End > Begin.
Size	End - Begin.	Нулевой или странный size связан с invalid_ranges.

Колонка	Что означает	Что проверять руками
Секция кода	Секция, куда попал Begin.	Обычно .text. Если не .text, нужен контекст.
Unwind RVA	RVA UNWIND_INFO.	Главный адрес metadata для проверки.
Unwind section	Секция UNWIND_INFO.	Обычно .xdata/.rdata/.pdata. Writable/executable/unusual - повод проверить.
Статус	OK или Ошибка.	Ошибка означает parse failure.
Version / flags	Версия и флаги UNWIND_INFO.	EHANDLER/UHANDLER/CHAININFO объясняют handler/chain behavior.
Stack	fixed_stack_bytes.	Сверять с prolog функции.
Handler	RVA handler, если есть.	Перейти и понять, это реальный обработчик, thunk или dispatcher.
Аномалии / ошибка	anomalies_text + parse_error.	Главное место для объяснения карточек `Ошибки разбора` и `Аномальные записи`.

45. JSON-элементы: где в JSON искать то же самое

HTML удобен человеку, JSON удобен проверкам и тестам. Одни и те же данные лежат в обоих форматах. Если надо доказать, что отчет не нарисован вручную, можно показать JSON: он повторяет поля HTML в строгой структуре.

JSON key	Соответствие в HTML	Что хранит
report_metadata	Служебные поля	Версия, hash, команда, build, strict_abi, follow_chain, max depth.
pe_validation	Структурная проверка	section_raw_oob, virtual_oob, overlaps, misaligned, exception_directory_oob.
sections	Таблица секций	name, virtual_address, virtual_size, raw_offset, raw_size, characteristics, readable/writable/executable.
heuristic_analysis	Карточки, статические признаки, weakening rules, details	score, suspicion, confidence, findings, evidence families, counters.
assessment	Итоговая оценка и ручной план	final_verdict, final_confidence, evidence levels, correlation, manual_validation_steps.
dynamic_validation	Динамическая проверка	enabled, ran, ok, events, runtime counters, trace, details. Есть только при dynamic data.
functions	Таблица RUNTIME_FUNCTION	index, begin/end, sections, unwind_ok/error, unwind summary, comment.
functions[].unwind	Version/flags/stack/handler/anomalies	version, flags, prolog_size, count_codes, fixed_stack_bytes, handler_rva, anomalies_text, chain.

46. Мини-алгоритм чтения отчета на защите

1. Сначала назвать объект анализа: путь, SHA-256, команда запуска.
2. Показать верхние карточки и объяснить, что каждая считает свой слой.
3. Если findings > 0, открыть `Статические признаки` и назвать code/family/severity/RVA.
4. Если dynamic включена, открыть `Динамическая проверка` и показать events.
5. Если надо доказать адрес, открыть `RUNTIME_FUNCTION` и перейти по Begin/Unwind/Handler RVA.
6. Завершить `Итоговой оценкой`: verdict, confidence, static/runtime levels, manual plan.

47. КОНТРОЛЬ ПОЛНОТЫ ПОЛЕЙ ОТЧЕТА

REPORT FIELD AUDIT CHECKLIST

Я сверил руководство с реальным HTML/JSON, схемой `docs/report.schema.json` и генератором `src/report.c`. Основные окна отчета уже были разобраны: верхние карточки, служебные поля, PE image, structural validation, sections, итоговая оценка, static findings, dynamic validation, таблица RUNTIME_FUNCTION и JSON-структура. Ниже добавлен точный словарь raw JSON-полей, которые аналитику полезно знать по именам.

Группа	Что уже объяснено	Что добавлено здесь
report_metadata/root PE	Происхождение отчета, hash, команда, image_base, alignment, exception directory.	Точный список служебных ключей и зачем они нужны для воспроизводимости.
heuristic_analysis/findings	Score, verdict, evidence, families, findings, weakening rules.	Точные поля finding: affected_count, representative_rva, representative_section, message.
dynamic_validation	События, runtime table calls, handlers, trace, dropped events.	Точные счетчики instrumentation/breakpoint/module coverage.
functions/unwind	RUNTIME_FUNCTION строки, Begin/End, Unwind RVA, handler, stack, anomalies.	Точные JSON-ключи begin_rva, end_rva, begin_section, unwind_error, unwind_section.

48. Служебные и PE-поля

Поле JSON	Что означает	Зачем аналитику
input	Путь к проверяемому файлу.	Понимать, какой объект анализировался.
report_metadata.schema_version	Версия JSON-контракта.	Понять, с какой схемой совместим отчет.
report_metadata.tool_version	Версия analyzer.	Воспроизводимость и сравнение отчетов.
report_metadata.generated_at_utc	Время создания отчета в UTC.	Отличать старый отчет от нового.
report_metadata.input_sha256	SHA-256 target-файла.	Доказать, что отчет относится именно к этому файлу.
report_metadata.input_size	Размер target-файла.	Sanity-check вместе с hash.
report_metadata.command_line	Команда запуска analyzer.	Повторить тот же отчет.
build_compiler/build_platform/build_mode	Чем и в каком режиме собран analyzer.	Понимать среду и provenance.
strict_abi/follow_chain/max_chain_depth	Параметры строгости и обработки chained unwind.	Понимать, почему отчет мог увидеть больше или меньше признаков.
image_base/size_of_image/size_of_headers	Ключевые поля PE Optional Header.	Проверять адреса и границы образа.
section_alignment/file_alignment	Выравнивание секций в памяти и файле.	Объяснять misalignment и raw/virtual mapping.
exception_directory.rva/size	Где лежит PE exception directory.	Начальная точка для таблицы RUNTIME_FUNCTION.

49. PE validation и sections

Поле JSON	Что означает	Как читать
pe_validation.section_raw_oob	Секции выходят за raw file bounds.	Серьезный structural сигнал.
pe_validation.section_virtual_oob	Секции выходят за SizeOfImage.	Нарушение memory layout.
pe_validation.section_raw_overlaps	Raw-диапазоны секций пересекаются.	Подозрение на нестандартный/сломанный layout.
pe_validation.section_virtual_overlaps	Virtual ranges пересекаются.	Проблема mapping в памяти.

Поле JSON	Что означает	Как читать
pe_validation.section_misaligned	Нарушено file/section alignment.	Может быть ошибкой или ручным вмешательством.
pe_validation.exception_directory_oob	Exception directory выходит за допустимые границы.	RUNTIME_FUNCTION может читаться недостоверно.
sections[].name	Имя секции.	Сравнить с обычными .text/.pdata/.xdata/.rdata.
sections[].virtual_address/virtual_size	Диапазон секции в RVA.	Понять, куда попадает Begin/Unwind/Handler RVA.
sections[].raw_offset/raw_size	Где секция лежит в файле.	Проверка в hex/editor.
sections[].characteristics	Флаги секции.	Источник R/W/X permissions.
sections[].readable/writable/executable	Расшифровка прав доступа.	Writable unwind или non-exec handler сразу видны.

50. Heuristic analysis и finding

Эти поля нужны, чтобы не спорить по одной красивой карточке. Аналитик должен смотреть не только score, а какие families сработали, сколько объектов затронуто и где есть representative RVA.

Поле JSON	Что означает	Как использовать
heuristic_analysis.score/suspicion/confidence	Итоговая статическая оценка, нормированная подозрительность и уверенность.	Score - triage, не доказательство malware.
requires_manual_review/verdict/evidence_level	Нужна ли ручная проверка, текстовый вывод, сила evidence.	Главная формулировка для отчета.
weak_finding_count/medium_finding_count/strong_finding_count	Сколько признаков по severity.	Отличить один weak сигнал от набора strong сигналов.
total	Общее число RUNTIME_FUNCTION записей.	Масштаб объекта.
pe_* counters	Счетчики PE structural findings.	Проверка layout до unwind-логики.
rf_invalid_ranges/rf_oob/rf_unwind_oob/rf_unsorted/rf_overlaps/rf_duplicate_begins	Проблемы таблицы RUNTIME_FUNCTION.	Показывают нарушение карты функций.
unwind_parse_fail/unwind_anomalies/unwind_abi_warnings	Ошибки декодирования, локальные аномалии, strict ABI warnings.	Разделять parser failure, anomaly и строгие ABI warning.
unwind_unusual_section/unwind_exec_section/unwind_writable_section	Где размещен UNWIND_INFO.	Главные признаки metadata placement.
handler_nonexec_section/handler_writable_section	Права секции handler target.	Handler должен быть объяснимым исполняемым кодом.
unwind_reuse_ratio/unwind_unique_ratio/handler_count/handler_reuse_ratio	Плотность переиспользования unwind/handler.	Искать central dispatcher или необычную концентрацию.
evidence_family_count/evidence_families	Сколько независимых групп evidence сработало.	Чем больше независимых family, тем сильнее case.
findings[].code/family/severity	Идентификатор, семейство и уровень finding.	То, что надо цитировать в защите.
findings[].affected_count	Сколько объектов затронуто finding.	Показывает масштаб признака.
findings[].representative_rva	Показательный RVA для проверки.	Точка входа в Ghidra/hex view.
findings[].representative_section	Секция representative RVA.	Сразу видно .text/.xdata/.rdata/нестандартную секцию.
findings[].message	Человеческое объяснение finding.	Быстрый смысл без чтения кода.
weakening_rules_applied/weakening_rule_count/details	Какие смягчающие правила применены и подробности.	Почему score мог быть ниже, чем ожидалось.

51. Dynamic validation: точные поля

Поле JSON	Что означает	Как читать
enabled/ran/ok/timed_out/target_exited/target_exit_code	Статус dynamic-сессии и завершения target.	Отличать не запущено, запущено успешно, timeout, crash/exit.
timeout_ms/trace_steps	Параметры dynamic запуска.	Понимать границы эксперимента.
exceptions_seen/first_chance/second_chance	Сколько исключений увидел debugger.	Показывает activity exception flow.
veh_registrations/veh_hits	VEH регистрации и попадания.	Runtime exception interception evidence.
vch_registrations/vch_hits	VCH регистрации и попадания.	Еще один слой exception callback.

Поле JSON	Что означает	Как читать
uef_set_calls/uef_hits	SetUnhandledExceptionFilter и срабатывания.	UEF-level exception behavior.
rtl_add_function_table_calls	Вызовы RtlAddFunctionTable.	Runtime registration of unwind table.
rtl_install_function_table_cb_calls	Вызовы RtlInstallFunctionTableCallback.	Callback-based runtime table.
unwind_handler_hits	Попадания в handler из UNWIND_INFO.	Связь static handler и runtime execution.
dropped_events	События, которые не удалось сохранить из-за лимитов.	Если больше 0, dynamic evidence неполное.
instrumentation_complete	Сборщик считает instrumentation полной.	Если false, вывод осторожнее.
modules_observed_peak	Максимум модулей, виденных в процессе.	Контекст полноты module mapping.
module_refresh_failures/module_list_truncations	Проблемы обновления или усечение списка модулей.	Может снижать доверие к module/section mapping.
unwind_handlers_discovered/unwind_handler_bp_set	Сколько handlers найдено и сколько breakpoint поставлено.	Покрытие handler monitoring.
breakpoint_set_attempts/breakpoint_set_failures	Попытки и ошибки установки breakpoint.	Если ошибки есть, часть runtime paths могла быть не видна.
breakpoint_rearm_failures/pending_rearm_failures	Проблемы повторной постановки breakpoint.	Влияет на полноту после первого hit.
breakpoint_capacity_drops	Сколько breakpoint не поставлено из-за лимита емкости.	Если больше 0, coverage ограничено.

52. Dynamic events, trace и functions

Поле JSON	Что означает	Как использовать
events[].type	Тип runtime-события.	Например exception, veh_register, rtl_add_function_table.
events[].thread_id	ID потока события.	Понимать, в каком thread был runtime signal.
events[].module	Модуль, к которому отнесено событие.	Связать адрес с DLL/EXE.
events[].exception_code	Код исключения.	Отличить breakpoint/single-step/access violation и т.п.
events[].first_chance	First-chance или second/n/a.	Понять стадию exception dispatch.
events[].address	Главный адрес события.	Переход в debugger/Ghidra.
events[].arg0/arg1/arg2/arg3	Аргументы API/event.	Для Rtl* здесь лежат адреса таблиц, counts, callbacks и служебные параметры.
trace[]	Ограниченная RIP-трасса после handler hit.	Показывает направление выполнения, но не заменяет полный debugger trace.
functions[].index	Номер RUNTIME_FUNCTION.	Стабильная ссылка на строку.
functions[].begin_rva/end_rva/size	Диапазон функции.	Проверять границы кода.
functions[].begin_section	Секция BeginAddress.	Обычно .text; если нет, нужен контекст.
functions[].unwind_rva/unwind_section	Где лежит UNWIND_INFO.	Главное поле для metadata placement.
functions[].unwind_ok/unwind_error	Статус декодирования и причина ошибки.	Если unwind_ok false, читать unwind-поля осторожно.
functions[].unwind.version/flags/prolog_size/count_codes	Заголовок UNWIND_INFO.	Проверка ABI layout.
functions[].unwind.fixed_stack_bytes/frame_reg/frame_off	Оценка stack frame.	Понять, как восстанавливается stack.
functions[].unwind.has_handler/handler_rva/handler_section	Есть ли handler и где он лежит.	Проверять executable/writable/central handler.
functions[].unwind.has_chain/chain	Есть ли chained RUNTIME_FUNCTION.	Проверка связи chained metadata.
functions[].unwind.anomalies/anomalies_text	Локальные anomaly bits и текст.	Именно это считает карточка 'Аномальные записи'.
functions[].comment	Сводный комментарий analyzer по строке.	Быстрый human-readable итог по функции.

53. Достаточно ли отчета аналитику

Для первичного triage, объяснимого разбора PE/unwind metadata и подготовки защиты отчет содержит нужный минимум и важные проверочные поля: provenance, hash, command_line, PE layout, section permissions, structural validation, RUNTIME_FUNCTION, UNWIND_INFO decode, findings, score, assessment, manual validation plan и dynamic event timeline. Это правильно для defensive analyzer: отчет не делает вид, что он decompiler или malware oracle.

Что аналитику нужно	Есть в отчете	Комментарий
Воспроизводимость	Да: input_sha256, command_line, tool/schema/build metadata.	Можно доказать, какой файл и какой запуск породили отчет.
Адресная проверка	Да: RVA, sections, representative_rva, handler_rva, unwind_rva.	Есть точки входа для Ghidra/debugger.

Что аналитику нужно	Есть в отчете	Комментарий
Статические признаки	Да: findings, families, severity, affected_count, counters.	Вывод объяснимый, не черный ящик.
Динамические события	Да: events, counters, trace, instrumentation counters.	Ограничено конкретным запуском, что честно отражено.
Итоговая интерпретация	Да: assessment, confidence, case_strength, correlation, manual plan.	Это triage/validation conclusion, не судебное доказательство само по себе.
Что остается вручную	Disassembly, semantic reverse engineering, memory write tracing, full path exploration.	Это нормально: такие вещи делаются в Ghidra/debugger/VM по manual validation plan.

Итог: отчет выводит все ключевые данные, которые нужны аналитику для первичного вывода и дальнейшей ручной проверки. Правильная формулировка на защите: инструмент строит объяснимую evidence model по PE/unwind metadata и runtime events, а окончательный вывод аналитик подтверждает вручную по адресам и рекомендациям из отчета.

54. Контрольные raw key names

Эта строка нужна как машинно-читаемая шпаргалка: если проверять PDF поиском по точному имени поля, эти ключи тоже должны находиться без переноса.

```
pending_rearm_failures
unwind_handler_bp_set
```